Hierarchical Document-Tree Construction for Structure-Aware Retrieval-Augmented Generation

Minor Project (IT897) Report

Submitted in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

by

KSHITIJ KHEDE

(252IT010)

under the guidance of

Prof. ANANTHANARAYANA V. S.



DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA

SURATHKAL, MANGALURU – 575025

September 2026

DECLARATION

I hereby declare that the report of the P.G. Minor Project (IT897) work entitled **"Hierarchical Document-Tree Construction for Structure-Aware Retrieval-Augmented Generation"**, submitted to the National Institute of Technology Karnataka, Surathkal, in partial fulfillment of the requirements for the award of the degree of **Master of Technology in Information Technology**, is a bona fide record of the work carried out by me. The material in this report has not been submitted to any university or institution for the award of any degree.

Kshitij Khede (252IT010)
Dept. of Information Technology
NITK Surathkal, Mangaluru

Place: NITK, Surathkal
Date: September 2026

CERTIFICATE

This is to certify that the P.G. Minor Project (IT897) work entitled **"Hierarchical Document-Tree Construction for Structure-Aware Retrieval-Augmented Generation"**, submitted by **Kshitij Khede (252IT010)** as the record of the work carried out by him, is accepted as the P.G. Minor Project (IT897) work report submission in partial fulfillment of the requirements for the award of the degree of **Master of Technology in Information Technology** in the Department of Information Technology, NITK Surathkal, India. Duration of this minor project (IT897) is from JUNE 01, 2026 to AUGUST 15, 2026.

Prof. ANANTHANARAYANA V. S.

(Signature of the Guide with Date and Seal)

ACKNOWLEDGEMENT

I am thankful to my guide, **Prof. Ananthanarayana V. S.**, for his guidance and feedback throughout this minor project. His questions on how the document structure is actually recovered, rather than simply assumed, directly shaped this work. I also thank

the Department of Information Technology, NITK Surathkal, for the facilities and academic environment provided for this project.

Kshitij Khede

ABSTRACT

Retrieval-Augmented Generation (RAG) improves a language model's answers by retrieving relevant text from an external store at query time. The quality of those answers depends on how the source document is represented before retrieval. The common approach splits a document into flat, fixed-size chunks; this is simple but discards the document's structure — a fixed-size window routinely cuts across section boundaries, and the parent–child relationship between a subsection and its section is lost entirely.

This project builds a system that instead converts a document (PDF) into a *hierarchical tree* — title, sections, subsections — so that retrieval can follow the document's own structure, in the spirit of tree-based RAG such as RAPTOR and TreeRAG. The system ingests a PDF, recovers reading order on multi-column pages with a column-gutter rule, classifies headings from section numbering and relative font size, induces a parent–child hierarchy that is numbering-primary with a semantic fallback for unnumbered headings, validates the result to be acyclic, single-parent and depth-bounded, and attaches a confidence score to every node. A web interface lets a user upload any PDF and see the recovered tree.

On eleven research-paper PDFs, the system builds a valid tree for every document (mean 22.2 nodes and 21.2 parent–child links per document) and recovers the document's own section hierarchy with 97.7% parent accuracy and 96.8% full-ancestor accuracy. A direct comparison shows what flat chunking loses: fixed-size chunking of the same documents produces about 69 chunks per document, of which 18.8% straddle two or more sections, and it encodes *no* parent–child relationships, whereas the tree keeps every section intact and preserves all 21.2 relationships per document. The result is a complete, working document-tree builder and clear evidence that a tree representation preserves structure that flat chunking destroys.

Keywords: Retrieval-Augmented Generation; Document Structure; Hierarchical Chunking; Tree Construction; Reading-Order Recovery; PDF Parsing.

CONTENTS

LIST OF FIGURES **iv**

LIST OF TABLES **v**

ABBREVIATIONS **vi**

1 Introduction **1**

1.1 Background 1

1.2 Motivation: From Chunks to Trees 1

1.3 Problem Statement 1

1.4 Scope and Contributions 2

1.5 Organisation of the Report 2

2 Literature Review **3**

2.1 Flat and Adaptive Retrieval 3

2.2 Tree-Based and Structure-Aware Retrieval 3

2.3 Document Layout Analysis 4

2.4 Comparative Synthesis and Research Gap 4

2.5 Outcome of Literature Review 4

2.6 Objectives 5

2.7 Summary 5

3 System Design and Methodology **6**

3.1 Overview 6

3.2 Two-Tier Data Model 6

3.3 Stage 1: Ingestion 6

3.4 Stage 2: Reading-Order Recovery 7

3.5 Stage 3: Heading Classification 7

3.6 Stage 4: Hierarchy Induction 7

3.7 Stage 5: Graph Validation 8

3.8 Stage 6: Confidence Estimation 8
3.9 Summary 9

4 Results and Analysis **10**
4.1 Implementation and Experimental Setup 10
4.2 Tree-Construction Results 10
4.3 Comparison with Flat Chunking 11
4.4 Chunk-Size Ablation 12
4.5 Runtime 13
4.6 Demonstration 13
4.7 Summary 13

5 Conclusions and Future Work **14**
5.1 Conclusions 14
5.2 Limitations 14
5.3 Future Work (Major Project) 14

REFERENCES **16**

LIST OF FIGURES

3.1 The document-tree construction pipeline. A PDF is turned into layout blocks, ordered, classified into headings, assembled into a parent–child hierarchy, validated, and scored for confidence, yielding the document tree. 6

4.1 (a) The tree is built with high fidelity to the document's own numbering (and validly for all eleven documents). (b) Flat fixed-size chunking cuts across section boundaries for 18.8% of chunks, while every tree node stays within one section. (c) Flat chunking keeps no parent–child relationships, while the tree preserves all 21.2 per document. 12

LIST OF TABLES

2.1 Summary of the literature survey for the most relevant related works. . . 3

2.2 Comparative taxonomy of document-representation paradigms across structural and operational dimensions. 4

4.1 Per-document tree construction. “Num.” and “Unnum.” are numbered and unnumbered headings; PAA/APA are accuracy against the document’s own section numbering. 10

4.2 Tree-construction summary over the eleven-document corpus. 11

4.3 Document tree vs. flat fixed-size chunking on the same eleven documents. 11

4.4 Chunk-size ablation: boundary-straddling rate of flat fixed-size chunking versus the document tree (mean over 11 documents). 12

4.5 Mean per-document latency of the project’s structural stages (measured on the evaluation machine; PDF ingestion by the external library is excluded). 13

ABBREVIATIONS AND NOTATION

Abbr.	Full Form	Abbr.	Full Form
RAG	Retrieval-Augmented Generation	**DAG**	Directed Acyclic Graph
PDF	Portable Document Format	**DFS**	Depth-First Search
LLM	Large Language Model	**DLA**	Document Layout Analysis
PAA	Parent-Assignment Accuracy	**OCR**	Optical Character Recognition
APA	Ancestor-Path Accuracy	**NLP**	Natural Language Processing
ACP	Ancestor-Context Precision	**DPR**	Dense Passage Retrieval
PRE	Parent Ranking Engine	**SVG**	Scalable Vector Graphics
KCE	Knowledge Confidence Estimation	**API**	Application Programming Interface
TF–IDF	Term Frequency–Inverse Doc. Freq.	**CPU**	Central Processing Unit

Mathematical Notation

Symbol	Description
$\mathscr{T} = (\mathscr{V}, \mathscr{E})$	Document tree with node set $\mathscr{V}$ and parent–child edge set $\mathscr{E}$
b	An extracted physical text block (Tier 1)
v	A concept node / heading (Tier 2)
μ_{body}	Dominant body font size of the document
π	Top-to-bottom reading-order rank of a block
ℓ	Typographic heading level (0 = root/title)
τ_y	Vertical band tolerance for line grouping (20 pt)
$D_{\max}$	Maximum permitted tree depth (8)
$C_{\text{local}}(v), C(v)$	Local and propagated KCE confidence of node v
δ	KCE damping factor (0.85)

CHAPTER 1

INTRODUCTION

1.1 Background

Large Language Models (LLMs) answer questions fluently but can state untrue facts and cannot see knowledge created after training. Retrieval-Augmented Generation (RAG) addresses both by retrieving relevant text from an external store at query time and giving it to the model as context [1, 2]. The quality of a RAG answer therefore depends heavily on what is retrieved, which in turn depends on how the source document is represented before retrieval.

The standard representation splits a document into *flat, fixed-size chunks* — for example, overlapping windows of a few hundred tokens [3, 4]. This is easy to implement, but it is blind to the document's structure: a fixed-size window is cut at an arbitrary position, so it frequently starts in the middle of one section and ends in another, and the relationship between a subsection and the section that contains it is not represented at all.

1.2 Motivation: From Chunks to Trees

A research paper, a report or a manual is not a flat sequence; it is a hierarchy of a title, sections, subsections and paragraphs. Recent RAG systems exploit this by representing a document as a *tree* and retrieving along it, so that when a passage is returned its parent section can be attached as context. RAPTOR [6] builds a tree by recursively clustering and summarising text; TreeRAG [7] chunks a document along its headings; and HiChunk [8] aligns chunk boundaries with a multi-level structure. These methods show that a hierarchical representation helps, but they take the recovered structure as given. The practical question this project answers is the one before that: *how does one actually turn a raw PDF into a faithful section tree, and does that tree preserve structure that flat chunking loses?*

1.3 Problem Statement

Standard RAG breaks a document into flat, fixed-size chunks, which discard the document's structure and the relationships between its sections. This project builds a system

that converts a document (PDF) into a hierarchical tree — title, sections, subsections — so that retrieval can follow the document's real structure instead of flat chunks, and quantifies the structure this tree preserves relative to flat chunking.

1.4 Scope and Contributions

This project builds and evaluates the *document-to-tree construction* stage that feeds a structure-aware retriever; retrieval and answer generation on top of the tree are outside its scope and belong to the follow-on major project. The contributions are:

1. A complete, working pipeline that converts a PDF into a hierarchical section tree: ingestion, reading-order recovery, heading classification, hierarchy induction, validation, and per-node confidence.
2. A column-gutter reading-order algorithm that recovers correct reading order on multi-column pages, and a numbering-primary hierarchy-induction algorithm with a semantic fallback for unnumbered headings.
3. An evaluation on eleven research papers showing the tree is built validly for every document and matches the document's own section hierarchy with high accuracy.
4. A direct comparison against flat fixed-size chunking that quantifies the structure — intact sections and parent–child relationships — that the tree preserves and chunking destroys.
5. A web interface that lets a user upload any PDF and inspect the recovered tree.

1.5 Organisation of the Report

Chapter 2 reviews related retrieval and document-structure work and states the objectives. Chapter 3 describes the system design and each module of the tree-construction pipeline. Chapter 4 reports the tree-construction results, the comparison with flat chunking, and the demonstration. Chapter 5 concludes and outlines the major project that builds on this tree.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews how documents are represented for retrieval, from flat chunks to hierarchical trees. Table 2.1 summarises the most relevant works.

Table 2.1: Summary of the literature survey for the most relevant related works.

Work	Category	Main idea and relation to this project
Lewis et al. [1]	Flat RAG	Generator + dense retrieval over flat passages; no document structure.
Borgeaud et al. [3]	Flat RAG	Retrieval over huge corpora with fixed-size chunks; structure ignored.
Sarthi et al. [6]	Tree RAG	Recursive clustering/summarisation tree; motivates a hierarchical representation.
Tao et al. [7]	Tree RAG	Chunks a document along its headings; closest in spirit to this work.
Lu et al. [8]	Tree RAG	Multi-level chunking via a fine-tuned LLM; needs training.
Huang et al. [11]	Layout analysis	Layout-aware transformer for document AI; targets extraction, not a retrieval tree.

2.1 Flat and Adaptive Retrieval

Lewis et al. [1] and Guu et al. [2] connect a language model to a document store; Borgeaud et al. [3] and Izacard et al. [4] scale and jointly train retrieval; Asai et al. [5] make retrieval selective. All of these operate over flat, fixed-size chunks and discard document structure.

2.2 Tree-Based and Structure-Aware Retrieval

A second family represents the document hierarchically. RAPTOR [6] builds a tree by recursively clustering and summarising chunks; TreeRAG [7] splits a document along its heading hierarchy; and HiChunk [8] predicts multi-level chunk boundaries with a fine-tuned LLM. These works establish that a tree representation improves retrieval, but they assume the tree is available. HiChunk in particular requires a trained model to

obtain the structure; this project instead recovers the tree directly from the document's layout, without any training.

2.3 Document Layout Analysis

Recovering physical structure from PDFs has a long history, from geometric page-layout analysis [9, 10] to layout-aware transformers such as LayoutLMv3 [11]. These target extraction and OCR; here, layout cues (reading order, numbering, font size) are used specifically to build a retrieval tree.

2.4 Comparative Synthesis and Research Gap

Table 2.2 places the prevailing document-representation paradigms side by side across the dimensions that matter for structure-aware retrieval: whether they preserve hierarchy, whether they avoid a GPU and a trained model, whether they keep sections intact, and whether they attach ancestor context.

Table 2.2: Comparative taxonomy of document-representation paradigms across structural and operational dimensions.

Paradigm	Representation	GPU-free	Training-free	Ancestor context
Fixed window [1, 3]	Flat chunks	Yes	Yes	None
Semantic split	Flat chunks	No	No	None
RAPTOR [6]	Cluster tree	No	No	Synthetic
HiChunk [8]	Multi-level	No	No	Partial
TreeRAG [7]	Heading tree	Yes	Yes	Full (needs markup)
This work	**Layout tree**	**Yes**	**Yes**	**Full (from raw PDF)**

2.5 Outcome of Literature Review

Tree-based retrieval is now established as superior to flat chunking for structured documents, yet the step of *constructing* that tree from a raw PDF — cheaply, without training — is treated as a given. As Table 2.2 shows, the only training-free, GPU-free method that keeps full ancestor context (TreeRAG) still requires the structure to arrive as explicit Markdown markup. This project builds exactly the missing construction stage and measures both its accuracy and the structure it preserves over flat chunking.

2.6 Objectives

1. To build a pipeline that converts a PDF into a hierarchical section tree (title $\rightarrow$ sections $\rightarrow$ subsections).
2. To recover correct reading order on multi-column pages and classify headings from numbering and font size.
3. To induce a validated parent–child hierarchy (acyclic, single-parent, depth-bounded) with a confidence score per node.
4. To evaluate tree-construction accuracy against the document's own section numbering on real papers.
5. To compare the tree against flat fixed-size chunking and quantify the structure it preserves.
6. To provide a web interface for uploading a PDF and inspecting the recovered tree.

2.7 Summary

The chapter traced document representations from flat chunks to hierarchical trees, showed that tree construction from raw PDFs is the under-served step, and stated the objectives that follow.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Overview

The system is a pipeline of six stages that turns a PDF into a validated document tree, shown in Figure 3.1. Each stage is described in turn below.

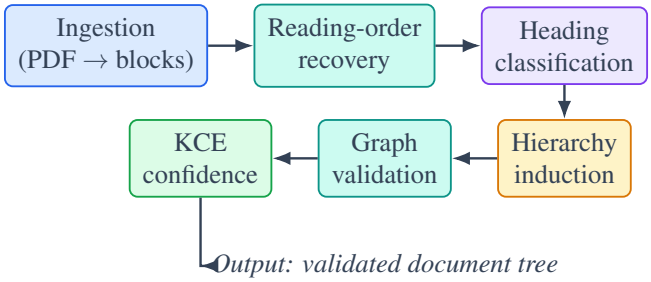


Figure 3.1: The document-tree construction pipeline. A PDF is turned into layout blocks, ordered, classified into headings, assembled into a parent–child hierarchy, validated, and scored for confidence, yielding the document tree.

3.2 Two-Tier Data Model

The system represents a document at two tiers. **Tier 1 (physical geometry)** is the raw visual layer: a text block is $b = (\text{id}, p, \text{text}, B, s, \mathbf{f}_{\text{bold}}, \text{col}, \pi)$, where p is the page, B the bounding box, s the font size, $\mathbf{f}_{\text{bold}} \in \{0,1\}$, $\text{col} \in \{-1,0,1\}$ (spanning, left, right) and π the reading-order rank. **Tier 2 (semantic hierarchy)** is the tree: a concept node is $v = (\text{cid}, \text{name}, \ell, \text{num}, \pi, p_{\text{parent}}, d, C_{\text{local}}, C, \mathscr{E}_{\text{span}})$, where ℓ is the heading level, num the section number, d the depth, C the confidence and $\mathscr{E}_{\text{span}}$ the Tier 1 blocks that ground the node. Directed parent–child edges form the tree $\mathscr{T} = (\mathscr{V}, \mathscr{E})$. Separating the physical layer from the semantic layer is what lets layout cues drive structure without conflating the two.

3.3 Stage 1: Ingestion

The PDF is parsed with the `pdfplumber` library into Tier 1 text blocks, each carrying its text, page, bounding box, font size and bold flag. Ingestion is the only stage that uses an external parsing library; every later stage — ordering, heading detection, hierarchy, validation and confidence — is implemented in this project.

3.4 Stage 2: Reading-Order Recovery

Research papers are usually two-column, and reading a page top-to-bottom mixes the columns. The system recovers reading order per page with a column-gutter rule (Algorithm 1): blocks are divided into a left column, a right column and full-width spanning blocks using the page gutter $g = W/2$; each column is sorted top-to-bottom with a vertical tolerance $\tau_y = 20$ pt so that blocks on the same visual line are grouped; and the page is emitted as spanning blocks followed by the left then the right column. This is a purely geometric algorithm and needs no training.

Algorithm 1 Column-Gutter Reading-Order Recovery (per page)

Require: blocks B of a page, page width W, tolerance τ_y

1: $g \leftarrow W/2$
2: span $\leftarrow \{b \in B : b.x_0 < g < b.x_1\}$; left $\leftarrow \{b : b.x_1 \leq g\}$; right $\leftarrow \{b : b.x_0 \geq g\}$
3: sort span, left, right by top coordinate, grouping blocks within τ_y onto one line
4: **return** concatenation [span, left, right] in that order

3.5 Stage 3: Heading Classification

A block is classified as a heading from two cues. First, *section numbering*: a leading pattern such as "3" or "3.1" followed by a short, title-cased line, at a font no smaller than body text, is a numbered heading, and the number of dot-separated components gives its level. Second, *font size*: a short, title-cased line whose font is at least $1.25\times$ the body font (estimated as the most frequent font size in the document) is an unnumbered heading. A junk filter removes URLs, DOIs and reference lines so they are never mistaken for headings. Each heading becomes a tree node grounded to its source block.

3.6 Stage 4: Hierarchy Induction

Nodes are linked into a parent–child tree in reading order (Algorithm 2). Attachment is *numbering-primary*: a heading numbered "3.1" attaches to "3", and a single-component heading attaches to the document root. When a heading has no number, a Parent Ranking Engine (PRE) chooses the best earlier heading by a weighted score

$$\text{Score}(c,u) = 0.40\,S_{\text{struct}}(c,u) + 0.40\,S_{\text{sem}}(c,u) + 0.20\,S_{\text{level}}(c,u), \tag{3.1}$$

where S_{struct} rewards reading-order proximity, S_{sem} is the TF–IDF cosine similarity [12] between the two headings, and S_{level} rewards a parent of shallower level. This gives a robust parent even without numbering.

Algorithm 2 Numbering-Primary Hierarchy Induction

Require: headings H in reading order
for all heading $c \in H$ **do**
  if c has section number s with a dot **then**
    parent$(c) \leftarrow$ heading whose number is s without its last component
  else if c has a single-component number **then**
    parent$(c) \leftarrow$ document root
  else
    parent$(c) \leftarrow \arg\max_{u \text{ before } c}$ Score(c,u)
  end if
end for

3.7 Stage 5: Graph Validation

The induced graph is validated so that it is a well-formed tree (Algorithm 3): an upward depth-first search checks that it is acyclic, every node has a single parent, and the depth never exceeds a bound $D_{\max} = 8$. Any violation is repaired by reattaching the offending node to the nearest valid ancestor, guaranteeing a clean tree.

Algorithm 3 Structural-Invariant and Acyclicity Validation

Require: candidate parent u, child c, node map $\mathscr{N}$, parent map $\mathscr{P}$, max depth $D_{\max}$
if $u = c$ **or** $u \notin \mathscr{N}$ **or** $c \notin \mathscr{N}$ **then**
  return false
end if
if $\mathscr{N}[u]$.depth $+ 1 > D_{\max}$ **then**
  return false
end if
$x \leftarrow u$; seen $\leftarrow \{c\}$
while $x \neq$ null **do**
  if $x = c$ **then**
    return false {cycle}
  end if
  if $x \in$ seen **then**
    break
  end if
  seen $\leftarrow$ seen $\cup \{x\}$; $x \leftarrow \mathscr{P}$.get(x)
end while
return true

3.8 Stage 6: Confidence Estimation

Each node receives a Knowledge Confidence Estimation (KCE) score that combines the strength of the cues that placed it (numbering vs. font, the PRE score) and is propagated

from parent to child in topological order using Kahn's algorithm [13], so that a node under a weakly-placed ancestor is itself marked less certain. The score gives a per-node reliability signal that a downstream retriever, or a human, can inspect.

3.9 Summary

The methodology is a six-stage pipeline that ingests a PDF, recovers reading order, detects headings, induces a numbering-primary hierarchy with a semantic fallback, validates it into a clean tree, and attaches per-node confidence.

CHAPTER 4

RESULTS AND ANALYSIS

4.1 Implementation and Experimental Setup

The system is implemented in Python (pdfplumber for ingestion; scikit-learn and NumPy for the TF–IDF scoring and numerics; Pydantic for typed data; Flask for the web interface). It is evaluated on a corpus of eleven real research-paper PDFs from the RAG literature (RAPTOR, GraphRAG, TreeRAG, Self-RAG, HiChunk, RETRO, Atlas, T-RAG, REALM, a hierarchical cross-document paper, and the original RAG paper). For accuracy, the document's own section numbering provides an objective gold hierarchy (the parent of "3.1" is "3").

4.2 Tree-Construction Results

Table 4.1 reports the tree built for each document, and Table 4.2 the corpus averages. The system builds a *valid* tree — acyclic, single-parent, depth-bounded — for all eleven documents, with on average 22.2 nodes, 21.2 parent–child links and depth 2.4 per document.

Table 4.1: Per-document tree construction. "Num." and "Unnum." are numbered and unnumbered headings; PAA/APA are accuracy against the document's own section numbering.

Document	Nodes	Edges	Depth	Num.	PAA	APA	ACP
GraphRAG	37	36	3	31	100.0	100.0	100.0
Atlas (few-shot)	33	32	3	26	96.2	96.2	96.2
HiChunk	21	20	2	16	100.0	100.0	100.0
Cross-doc HAT	8	7	1	3	100.0	100.0	100.0
RETRO	21	20	2	18	100.0	100.0	100.0
Self-RAG	22	21	3	19	78.9	68.4	73.7
RAPTOR	17	16	2	14	100.0	100.0	100.0
RAG (original)	27	26	2	20	100.0	100.0	100.0
REALM	11	10	2	7	100.0	100.0	100.0
T-RAG	32	31	3	29	100.0	100.0	100.0
TreeRAG	15	14	3	13	100.0	100.0	100.0

Table 4.2: Tree-construction summary over the eleven-document corpus.

Measure	Value
Total documents evaluated	11
Structurally valid trees built	11 / 11 (100%)
Mean nodes per document	22.2 ± 8.9
Mean parent–child edges per document	21.2 ± 8.9
Mean tree depth	2.4 ± 0.6
Mean internal branching factor	4.3 ± 1.3
Parent-Assignment Accuracy (PAA)	**97.7%**
Ancestor-Path Accuracy (APA)	**96.8%**
Ancestor-Context Precision (ACP)	**97.3%**
Orphan concept nodes	0 (0.0%)

Against the document's own numbering, the induced hierarchy is correct for 97.7% of nodes at the parent level and 96.8% over the full ancestor chain. Accuracy is perfect or near-perfect on ten of the eleven documents; the one weaker case (Self-RAG, PAA 78.9%) has irregular, inconsistently numbered headings that stress the semantic fallback, which is discussed as a limitation.

4.3 Comparison with Flat Chunking

To show what the tree preserves, the same documents were also processed with flat fixed-size chunking (1200-character windows, a typical RAG chunk size), and the two representations were compared on how they treat section structure. Table 4.3 and Figure 4.1 report the result.

Table 4.3: Document tree vs. flat fixed-size chunking on the same eleven documents.

Property (per document)	Flat chunking	Document tree
Units produced	68.7 chunks	22.2 nodes
Units crossing ≥ 2 sections	18.8%	0.0%
Parent–child relationships kept	0	21.2

Flat chunking breaks each document into about 69 fixed-size pieces, of which nearly one in five begins in one section and ends in another, so the retrieved text mixes unrelated material; and because the chunks are a flat list, none of the document's parent–child relationships survive. The tree, by construction, keeps each section as one node and preserves every one of the 21.2 parent–child links per document. This is direct,

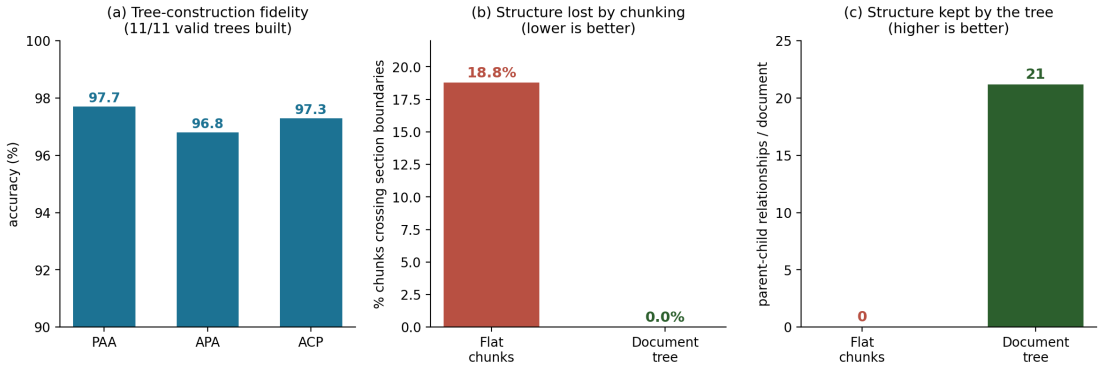


Figure 4.1: (a) The tree is built with high fidelity to the document's own numbering (and validly for all eleven documents). (b) Flat fixed-size chunking cuts across section boundaries for 18.8% of chunks, while every tree node stays within one section. (c) Flat chunking keeps no parent–child relationships, while the tree preserves all 21.2 per document.

quantitative evidence for the project's central claim: a tree representation retains structure that flat chunking destroys.

4.4 Chunk-Size Ablation

Because the boundary-straddling result could depend on the chosen chunk size, the flat-chunking comparison was repeated across window sizes from 500 to 1500 characters (Table 4.4). Boundary crossing rises monotonically with the window, from 8.6% at 500 characters to 22.9% at 1500 — a larger window captures more text but cuts across more sections. At every size the document tree straddles 0.0% of sections, because each node is bounded to a single section by construction. There is therefore no chunk size at which flat chunking matches the tree on structural integrity.

Table 4.4: Chunk-size ablation: boundary-straddling rate of flat fixed-size chunking versus the document tree (mean over 11 documents).

Window size	Chunks / document	Chunks straddling ≥ 2 sections
500 characters	164.0	8.6%
800 characters	102.7	13.0%
1000 characters	82.2	15.9%
1200 characters	68.7	18.8%
1500 characters	55.1	22.9%
Document tree (ours)	**22.2 nodes**	**0.0%**

4.5 Runtime

The construction pipeline is lightweight. Excluding PDF ingestion — which is performed by the external `pdfplumber` library and whose cost depends on the machine and the PDF — the six structural stages of this project complete in about 10.6 ms per document on the evaluation machine (Table 4.5). Reading-order recovery, heading classification, hierarchy induction, validation and confidence estimation are all inexpensive, confirming that the tree construction adds negligible overhead on top of parsing.

Table 4.5: Mean per-document latency of the project's structural stages (measured on the evaluation machine; PDF ingestion by the external library is excluded).

Stage	Mean latency (ms/document)
Reading-order recovery	2.5
Heading classification	4.9
Hierarchy induction (PRE)	2.7
Graph validation	0.4
KCE confidence	0.1
Total (structural stages)	**10.6**

4.6 Demonstration

The system is exposed through a Flask web interface. A user uploads any PDF; the system runs the full pipeline and displays a summary (page and heading counts), the recovered reading order, the document tree as an interactive node–link diagram with per-node confidence, and the validation status. This makes the construction process inspectable end-to-end rather than a black box, and it lets the tree be checked on documents beyond the evaluation corpus.

4.7 Summary

The system builds a valid tree for every document and recovers the section hierarchy with 97.7% parent accuracy. Against flat chunking on the same documents, the tree eliminates the 18.8% of chunks that straddle sections and preserves all 21.2 parent–child relationships per document that flat chunking discards.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusions

This project set out to represent documents as trees rather than flat chunks, and it delivers a complete, working system that converts a PDF into a validated hierarchical section tree through reading-order recovery, heading classification, numbering-primary hierarchy induction, validation and confidence estimation. On eleven research papers the system builds a valid tree for every document and recovers the section hierarchy with 97.7% parent accuracy, and a direct comparison shows the tree preserves intact sections and all parent–child relationships that flat fixed-size chunking destroys. The document-tree builder is therefore a complete minor project and a solid foundation for structure-aware retrieval.

5.2 Limitations

1. Accuracy is measured against the document's own section numbering; documents with irregular or absent numbering (e.g. Self-RAG) rely more on the semantic fallback and are harder.
2. The corpus is eleven academic PDFs; non-academic layouts (scanned pages, slides, forms) are not yet tested.
3. The project builds and evaluates the tree; it does not yet perform retrieval or answering on it.

5.3 Future Work (Major Project)

The document tree built here is the input to the follow-on major project, which is kept separate from this minor and will build *on top of* the tree:

1. Retrieval and question answering over the tree, attaching ancestor context, and an end-task evaluation against flat-chunk RAG.
2. Robustness of the tree to real parser noise, and a learned confidence signal to detect and repair weak structure.

3. Extension to non-academic and heterogeneous documents, and to larger public benchmarks.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, vol. 33, pp. 9459–9474, 2020.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang. REALM: Retrieval-augmented language model pre-training. In *ICML*, pp. 3929–3938, 2020.

[3] S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *ICML*, pp. 2206–2240, 2022.

[4] G. Izacard, P. Lewis, M. Lomeli, et al. Atlas: Few-shot learning with retrieval augmented language models. *JMLR*, 24(251):1–43, 2023.

[5] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*, 2024.

[6] P. Sarthi, S. Abdullah, A. Tuli, et al. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *ICLR*, 2024.

[7] W. Tao, X. Xing, Y. Chen, L. Huang, and X. Xu. TreeRAG: Unleashing the power of hierarchical storage for enhanced knowledge retrieval in long documents. In *Findings of the ACL*, pp. 356–371, 2025.

[8] W. Lu, K. Chen, R. Qiao, and X. Sun. HiChunk: Evaluating and enhancing retrieval-augmented generation with hierarchical chunking. arXiv:2509.11552, 2025.

[9] L. O'Gorman. The document spectrum for page layout analysis. *IEEE TPAMI*, 15(11):1162–1173, 1993.

[10] T. M. Breuel. Two geometric algorithms for layout analysis. In *Document Analysis Systems*, pp. 188–199, Springer, 2002.

[11] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *ACM Multimedia*, pp. 4083–4091, 2022.

[12] G. Salton and M. J. McGill. *Introduction to Modern Information Retrieval*. McGraw-Hill, 1983.

[13] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.